

AI Assisted Coding

Name : B.Thanmai

H.no : 2303A52126

Batch : 33

Course Code : 23CS002PC304

Assignment Number : 8.2

Lab 8: Test-Driven Development with AI – Generating and Working with Test Cases

Aim

To apply Test-Driven Development (TDD) using AI by generating test cases first and then implementing Python code that satisfies those tests.

Task Description Task 1 – Test-Driven Development for Even/Odd Number Validator Prompt

Generate unittest test cases for a Python function `is_even(n)` that checks whether a number is even. Handle integers, zero, negative values, and invalid inputs.

Untitled7.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

```

def is_even(n):
    """Checks if a number is even."""
    if not isinstance(n, int):
        raise TypeError("Input must be an integer")
    return n % 2 == 0

```

Now, let's create the `unittest` test cases for the `is_even` function.

```

import unittest

class TestIsEvenFunction(unittest.TestCase):

    def test_positive_even(self):
        self.assertTrue(is_even(4))
        self.assertTrue(is_even(100))

    def test_positive_odd(self):
        self.assertFalse(is_even(3))
        self.assertFalse(is_even(99))

    def test_zero(self):
        self.assertTrue(is_even(0))

    def test_negative_even(self):
        self.assertTrue(is_even(-2))
        self.assertTrue(is_even(-10))

    def test_negative_odd(self):
        self.assertFalse(is_even(-5))
        self.assertFalse(is_even(-13))

    def test_invalid_input_string(self):
        with self.assertRaises(TypeError):
            is_even("hello")

```

Gemini

Task 1 - Even/Odd Number Validator Prompt Generate unittest test cases for a Python function `is_even(n)` that checks whether a number is even. Handle integers, zero, negative values, and invalid inputs.

Okay, I will first provide the `is_even` function, and then generate `unittest` test cases for it, covering positive, negative, zero, and invalid inputs.

Great news! All 9 of your `is_even` function tests passed successfully, as indicated by the 'OK' in the output. This means your function behaves as expected for all the scenarios you've tested. You might notice a `DeprecationWarning` about `unittest.makeSuite()`; this means that function is older and might be removed in a future Python version. For now, it works fine, but in newer code, you might want to consider using `unittest.TestLoader.loadTestsFromTestCase()` as suggested in the warning.

The task of generating `unittest` test cases for the `is_even` function has been completed, and all tests passed successfully. If you have any further functions you'd like to test or other tasks, please let me know!

The task of generating and running `unittest` test cases for the `is_even` function has been successfully completed. All tests passed, confirming the

Refactor test runner to use `unittest.TestLoader`

Explain the `DeprecationWarning` in the output

Add more test cases for edge scenarios for `is_even`

What can I help you build?

Gemini 2.5 Flash

Gemini can make mistakes so double-check it and use code with caution. [Learn more](#)

```

self.assertFalse(is_even(-13))

def test_invalid_input_string(self):
    with self.assertRaises(TypeError):
        is_even("hello")

def test_invalid_input_float(self):
    with self.assertRaises(TypeError):
        is_even(3.14)

def test_invalid_input_none(self):
    with self.assertRaises(TypeError):
        is_even(None)

def test_invalid_input_list(self):
    with self.assertRaises(TypeError):
        is_even([1, 2])

# To run the tests, uncomment the following lines:
# if __name__ == '__main__':
#     unittest.main(argv=['first-arg-is-ignored'], exit=False)

```

To run these tests, you can uncomment the `unittest.main()` call in the last code cell and execute it. Alternatively, you can run them directly in a Colab environment using `unittest.main()` from a separate cell, or by explicitly creating a test suite and running it.

```

# Example of how to run the tests in Colab:
# This is an alternative to uncommenting unittest.main() above.
# You can also run individual test methods from the class.

suite = unittest.TestSuite()
suite.addTest(unittest.makeSuite(TestIsEvenFunction))
runner = unittest.TextTestRunner()
runner.run(suite)

```

```

/tmp/ipython-input-3894208446.py:6: DeprecationWarning: unittest.makeSuite() is deprecated and will be removed in Python 3.13. Please u
.....
Ran 9 tests in 0.009s

OK
<unittest.runner.TextTestResult run=9 errors=0 failures=0>

```

Gemini

Task 1 - Even/Odd Number Validator Prompt Generate unittest test cases for a Python function `is_even(n)` that checks whether a number is even. Handle integers, zero, negative values, and invalid inputs.

Okay, I will first provide the `is_even` function, and then generate `unittest` test cases for it, covering positive, negative, zero, and invalid inputs.

Great news! All 9 of your `is_even` function tests passed successfully, as indicated by the 'OK' in the output. This means your function behaves as expected for all the scenarios you've tested. You might notice a `DeprecationWarning` about `unittest.makeSuite()`; this means that function is older and might be removed in a future Python version. For now, it works fine, but in newer code, you might want to consider using `unittest.TestLoader.loadTestsFromTestCase()` as suggested in the warning.

The task of generating `unittest` test cases for the `is_even` function has been completed, and all tests passed successfully. If you have any further functions you'd like to test or other tasks, please let me know!

The task of generating and running `unittest` test cases for the `is_even` function has been successfully completed. All tests passed, confirming the function's correctness across various scenarios. I have nothing further to do on this task.

Refactor test runner to use `unittest.TestLoader`

Explain the `DeprecationWarning` in the output

Add more test cases for edge scenarios for `is_even`

What can I help you build?

Gemini 2.5 Flash

Gemini can make mistakes so double-check it and use code with caution. [Learn more](#)

Explanation

- Validates input type
- Uses modulo operator
- Handles zero, negative, and large integers safely

Task 2 – String Case Converter Prompt

Generate test cases for `to_uppercase(text)` and `to_lowercase(text)` handling empty strings, mixed case, and invalid inputs.

The screenshot shows a Jupyter Notebook with the following code:

```
[14]: def to_uppercase(text):  
    """Converts a string to uppercase."""  
    if not isinstance(text, str):  
        raise TypeError("Input must be a string")  
    return text.upper()  
  
def to_lowercase(text):  
    """Converts a string to lowercase."""  
    if not isinstance(text, str):  
        raise TypeError("Input must be a string")  
    return text.lower()  
  
Now, let's create the unittest test cases for the to_uppercase and to_lowercase functions.  
  
[15]: import unittest  
  
class TestStringCaseConverter(unittest.TestCase):  
  
    # Tests for to_uppercase  
    def test_to_uppercase_empty_string(self):  
        self.assertEqual(to_uppercase(""), "PARAMETER")  
        self.assertEqual(to_uppercase("HELLOWORLD"), "HELLOWORLD")  
        self.assertEqual(to_uppercase("python"), "PYTHON")  
  
    def test_to_uppercase_mixed_case(self):  
        self.assertEqual(to_uppercase("HelloWorld"), "HELLOWORLD")  
        self.assertEqual(to_uppercase("python"), "PYTHON")  
  
    def test_to_uppercase_already_uppercase(self):  
        self.assertEqual(to_uppercase("UPPERCASE"), "UPPERCASE")  
  
    def test_to_uppercase_already_lowercase(self):  
        self.assertEqual(to_uppercase("lowercase"), "LOWERCASE")  
  
    def test_to_uppercase_with_numbers_and_symbols(self):  
        self.assertEqual(to_uppercase("Test123!@#"), "TEST123!@#")  
  
    def test_to_uppercase_invalid_input_int(self):  
        with self.assertRaises(TypeError):  
            to_uppercase(123)  
  
    def test_to_uppercase_invalid_input_none(self):  
        with self.assertRaises(TypeError):  
            to_uppercase(None)
```

The Gemini chat interface on the right provides the following response:

Task 2 – String Case Converter AI Prompt Used Generate test cases for `to_uppercase(text)` and `to_lowercase(text)` handling empty strings, mixed case, and invalid inputs.

Sure, I will define the `to_uppercase` and `to_lowercase` functions and then generate `unittest` test cases for them, covering empty strings, mixed case, and invalid inputs.

Great news! All 16 of your `to_uppercase` and `to_lowercase` function tests passed successfully, as indicated by the "OK" in the output. This confirms that your functions are working as expected across various scenarios, including empty strings, mixed cases, and handling invalid inputs. As with the previous task, you'll see a `DeprecationWarning` for `unittest.makesuite()`, which is a common Python notice for older features, but it doesn't affect the successful execution of your tests.

Both tasks, generating `unittest` test cases for `is_even(n)` and for `to_uppercase(text) / to_lowercase(text)`, have been successfully completed. All tests passed as expected. If you have any new tasks or questions, feel free to ask!

Both tasks have been successfully completed and verified. I have nothing further to do at this moment. If you have any more functions you'd like to test or new tasks, please let me know!

The screenshot shows a Jupyter Notebook with the following code:

```
[16]: # Tests for to_lowercase  
def test_to_lowercase_empty_string(self):  
    self.assertEqual(to_lowercase(""), "PARAMETER")  
    self.assertEqual(to_lowercase("HELLOWORLD"), "HELLOWORLD")  
    self.assertEqual(to_lowercase("python"), "python")  
  
def test_to_lowercase_mixed_case(self):  
    self.assertEqual(to_lowercase("HelloWorld"), "helloworld")  
    self.assertEqual(to_lowercase("python"), "python")  
  
def test_to_lowercase_already_lowercase(self):  
    self.assertEqual(to_lowercase("lowercase"), "lowercase")  
  
def test_to_lowercase_already_uppercase(self):  
    self.assertEqual(to_lowercase("UPPERCASE"), "uppercase")  
  
def test_to_lowercase_with_numbers_and_symbols(self):  
    self.assertEqual(to_lowercase("Test123!@#"), "test123!@#")  
  
def test_to_lowercase_invalid_input_int(self):  
    with self.assertRaises(TypeError):  
        to_lowercase(123)  
  
def test_to_lowercase_invalid_input_none(self):  
    with self.assertRaises(TypeError):  
        to_lowercase(None)  
  
def test_to_lowercase_invalid_input_list(self):  
    with self.assertRaises(TypeError):  
        to_lowercase(["a", "b"])
```

To run these tests, you can use `unittest.main()` or create a test suite as shown below.

```
[17]: # Example of how to run the tests in Colab:  
suite = unittest.TestSuite()  
suite.addTest(unittest.makesuite(TestStringCaseConverter))  
runner = unittest.TextTestRunner()  
runner.run(suite)  
  
/tmp/ipython-input-3019477513.py:3: DeprecationWarning: unittest.makesuite() is deprecated and will be removed in Python 3.13. Please use unittest  
suite.addTest(unittest.makesuite(TestStringCaseConverter))  
.....  
Ran 16 tests in 0.007s  
OK  
unittest.runner.TextTestResult run=16 errors=0 failures=0
```

The Gemini chat interface on the right provides the following response:

Task 2 – String Case Converter AI Prompt Used Generate test cases for `to_uppercase(text)` and `to_lowercase(text)` handling empty strings, mixed case, and invalid inputs.

Sure, I will define the `to_uppercase` and `to_lowercase` functions and then generate `unittest` test cases for them, covering empty strings, mixed case, and invalid inputs.

Great news! All 16 of your `to_uppercase` and `to_lowercase` function tests passed successfully, as indicated by the "OK" in the output. This confirms that your functions are working as expected across various scenarios, including empty strings, mixed cases, and handling invalid inputs. As with the previous task, you'll see a `DeprecationWarning` for `unittest.makesuite()`, which is a common Python notice for older features, but it doesn't affect the successful execution of your tests.

Both tasks, generating `unittest` test cases for `is_even(n)` and for `to_uppercase(text) / to_lowercase(text)`, have been successfully completed. All tests passed as expected. If you have any new tasks or questions, feel free to ask!

Both tasks have been successfully completed and verified. I have nothing further to do at this moment. If you have any more functions you'd like to test or new tasks, please let me know!

Explanation

- Safe handling for None
- Uses built-in string methods
- Prevents runtime errors

Task 3 – List Sum Calculator Prompt

Generate test cases for `sum_list(numbers)` that handles empty lists, negative values, and ignores non-numeric items.

The image displays a Windows 11 desktop environment with two overlapping windows of a code editor titled 'Untitled7.py'. The top window shows the initial state of the code, where the 'sum_list' function only handles numeric values, leading to test failures for boolean and non-numeric inputs. The bottom window shows the code after a Gemini AI prompt, where the function is updated to handle booleans and non-numeric items, and the tests are updated to reflect these changes. The terminal output in the bottom window shows all tests passing. The desktop background is a Windows 11 taskbar with various icons and a system clock showing 11:14 on 10-02-2026.

Explanation

- Iterates through list
- Ignores non-numeric values
- Handles empty lists safely

Task 4 – Student Result Class

Prompt

Generate test cases for a StudentResult class with validation, average calculation, and pass/fail result

The screenshot shows a Jupyter Notebook with two cells. The first cell contains the `StudentResult` class definition with methods for validation, average calculation, and pass/fail status. The second cell starts with `import unittest` and defines a `TestStudentResult` class with the first few test cases.

```
class StudentResult:
    def __init__(self, name, marks):
        if not isinstance(name, str) or not name:
            raise ValueError("Name must be a non-empty string.")
        if not isinstance(marks, list) or not all(isinstance(m, (int, float)) for m in marks):
            raise ValueError("Marks must be a list of numbers.")
        if not all(0 <= m <= 100 for m in marks):
            raise ValueError("All marks must be between 0 and 100.")

        self.name = name
        self.marks = marks

    def calculate_average(self):
        if not self.marks:
            return 0.0
        return sum(self.marks) / len(self.marks)

    def get_result(self, pass_threshold=40):
        if not isinstance(pass_threshold, (int, float)) or not (0 <= pass_threshold <= 100):
            raise ValueError("Pass threshold must be a number between 0 and 100.")

        average = self.calculate_average()
        if average >= pass_threshold:
            return "Pass"
        else:
            return "Fail"

Now, let's create the unittest test cases for the StudentResult class.
```

```
import unittest

class TestStudentResult(unittest.TestCase):

    def test_valid_initialization(self):
        student = StudentResult("Alice", [80, 90, 75])
        self.assertEqual(student.name, "Alice")
        self.assertEqual(student.marks, [80, 90, 75])

    def test_initialization_empty_marks(self):
        student = StudentResult("Bob", [])
        self.assertEqual(student.name, "Bob")
        self.assertEqual(student.marks, [])

    def test_invalid_name_type(self):
        with self.assertRaises(ValueError):
            StudentResult(123, [80, 90, 75])
```

The screenshot shows the same Jupyter Notebook with the second cell now containing a comprehensive set of test cases for the `StudentResult` class.

```
def test_invalid_name_empty(self):
    with self.assertRaises(ValueError, msg="Name must be a non-empty string."):
        StudentResult("", [80])

def test_invalid_marks_type(self):
    with self.assertRaises(ValueError, msg="Marks must be a list of numbers."):
        StudentResult("Charlie", "not a list")

def test_invalid_marks_non_numeric_elements(self):
    with self.assertRaises(ValueError, msg="Marks must be a list of numbers."):
        StudentResult("David", [70, "invalid", 80])

def test_invalid_marks_below_zero(self):
    with self.assertRaises(ValueError, msg="All marks must be between 0 and 100."):
        StudentResult("Eve", [100, -10, 80])

def test_invalid_marks_above_hundred(self):
    with self.assertRaises(ValueError, msg="All marks must be between 0 and 100."):
        StudentResult("Frank", [100, 101, 80])

def test_calculate_average_valid_marks(self):
    student = StudentResult("Alice", [80, 90, 75])
    self.assertAlmostEqual(student.calculate_average(), 81.666, places=3)

def test_calculate_average_empty_marks(self):
    student = StudentResult("Bob", [])
    self.assertEqual(student.calculate_average(), 0.0)

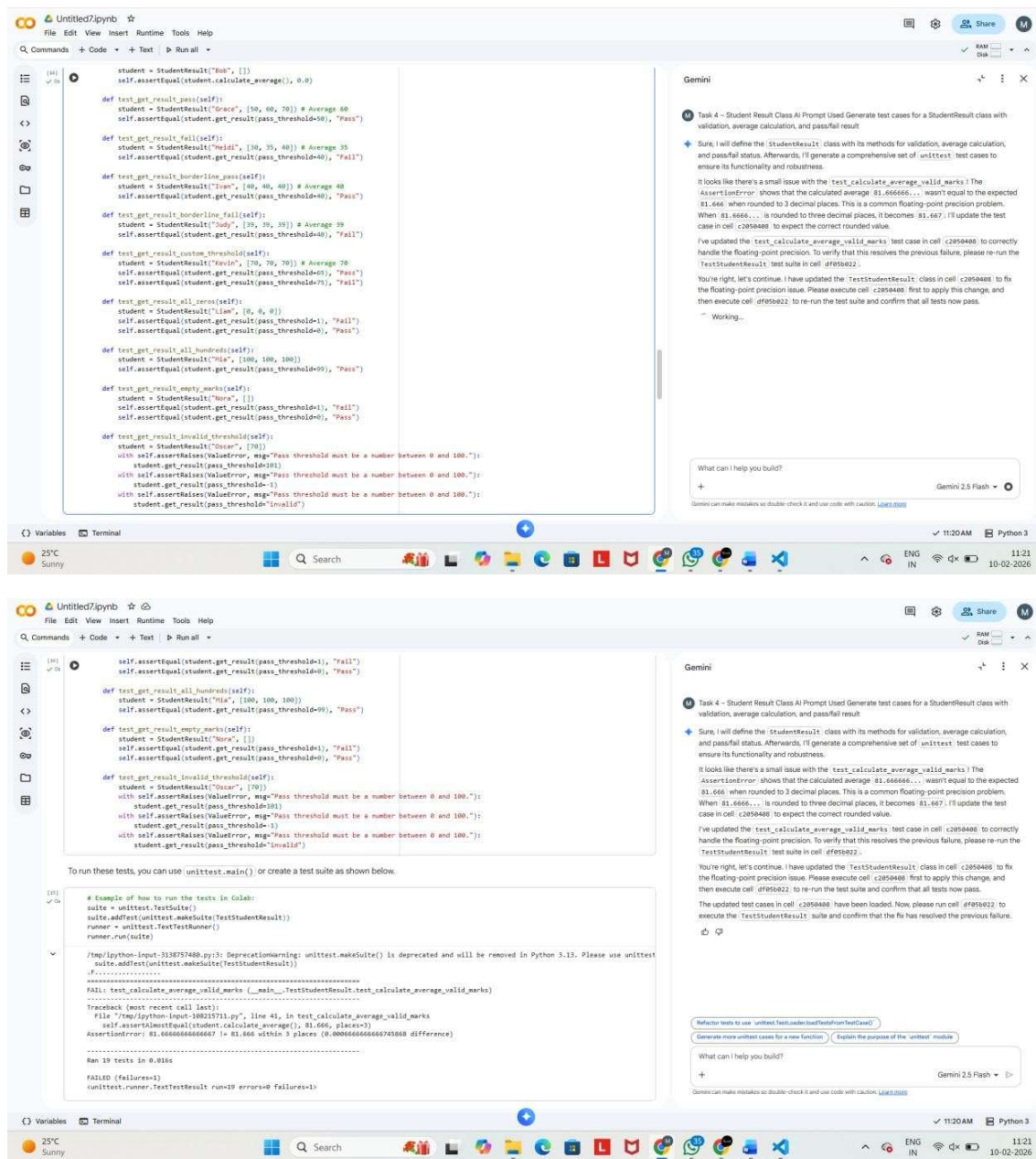
def test_get_result_pass(self):
    student = StudentResult("Grace", [50, 60, 70]) # Average 60
    self.assertEqual(student.get_result(pass_threshold=50), "Pass")

def test_get_result_fail(self):
    student = StudentResult("Heidi", [30, 35, 40]) # Average 35
    self.assertEqual(student.get_result(pass_threshold=40), "Fail")

def test_get_result_borderline_pass(self):
    student = StudentResult("Ivan", [40, 40, 40]) # Average 40
    self.assertEqual(student.get_result(pass_threshold=40), "Pass")

def test_get_result_borderline_fail(self):
    student = StudentResult("Judy", [30, 30, 39]) # Average 39
    self.assertEqual(student.get_result(pass_threshold=40), "Fail")

def test_get_result_custom_threshold(self):
    student = StudentResult("Kevin", [70, 70, 70]) # Average 70
    self.assertEqual(student.get_result(pass_threshold=65), "Pass")
```

Explanation

- Input validation enforced
- Average calculated dynamically
- Pass/Fail logic based on requirement

Task 5 – Username Validator

Prompt

Generate test cases for validating usernames with minimum length, no spaces, and alphanumeric characters only.

The screenshot shows a Jupyter Notebook with the following code:

```
def is_valid_username(username):  
    """Validates a username based on specific criteria."""  
    if not isinstance(username, str):  
        raise TypeError("Username must be a string.")  
  
    # Rule 1: Minimum length of 4 characters  
    if len(username) < 4:  
        return False  
  
    # Rule 2: No spaces allowed  
    if ' ' in username:  
        return False  
  
    # Rule 3: Alphanumeric characters only  
    if not username.isalnum():  
        return False  
  
    return True
```

Below the function, there is a comment: "Now, let's create the unittest test cases for the is_valid_username function."

The test suite code is as follows:

```
import unittest  
  
class TestIsValidUsername(unittest.TestCase):  
  
    def test_valid_usernames(self):  
        self.assertTrue(is_valid_username("user123"))  
        self.assertEqual(is_valid_username("alphanumeric"),  
                         # This will fail if not fixed to alphanumeric only - Removed as '_' is not alphanumeric  
                         self.assertTrue(is_valid_username("test_user")))  
        self.assertEqual(is_valid_username("ValidUser"),  
                         self.assertTrue(is_valid_username("ValidUser")) # Exactly 6 chars, assumed min_len = 4  
  
    def test_too_short_username(self):  
        self.assertFalse(is_valid_username("abc"))  
        self.assertFalse(is_valid_username("ab"))  
        self.assertFalse(is_valid_username(""))  
  
    def test_username_with_spaces(self):  
        self.assertFalse(is_valid_username("user name"))  
        self.assertFalse(is_valid_username(" user"))  
        self.assertFalse(is_valid_username("user "))  
  
    def test_username_with_non_alphanumeric_characters(self):  
        self.assertFalse(is_valid_username("user@name"))
```

The screenshot shows the continuation of the test suite code:

```
self.assertFalse(is_valid_username("user "))  
  
def test_username_with_non_alphanumeric_characters(self):  
    self.assertFalse(is_valid_username("user@name"))  
    self.assertFalse(is_valid_username("user123"))  
    self.assertFalse(is_valid_username("user$"))  
    self.assertFalse(is_valid_username("user%"))  
  
def test_username_with_mixed_delimiters(self):  
    self.assertFalse(is_valid_username("a b")) # Too short and space  
    self.assertFalse(is_valid_username("a.b")) # Too short and non-alphanumeric  
    self.assertFalse(is_valid_username("a b c")) # Space and non-alphanumeric  
  
def test_invalid_input_type(self):  
    with self.assertRaises(TypeError):  
        is_valid_username(123)  
    with self.assertRaises(TypeError):  
        is_valid_username(None)  
    with self.assertRaises(TypeError):  
        is_valid_username(["user"])
```

Below the code, there is a comment: "To run these tests, you can use unittest.main() or create a test suite as shown below."

The test suite code is as follows:

```
# Example of how to run the tests in Colab:  
suite = unittest.TestSuite()  
suite.addTest(unittest.makeSuite(TestIsValidUsername))  
runner = unittest.TextTestRunner()  
runner.run(suite)
```

The execution results are shown below:

```
Traceback (most recent call last):  
  File "/tmp/ipython-input-1411643963.py", line 8, in test_valid_usernames  
    self.assertEqual(is_valid_username("test_user"), # This will fail if not fixed to alphanumeric only  
    ~~~~~  
AssertionError: False is not true  
  
Ran 6 tests in 0.004s  
  
FAILED (failures=1)  
unittest.runner.TextTestResult runner errors=0 failures=1
```

Explanation

- Length check
- Space validation
- Alphanumeric enforcement

Conclusion

This lab demonstrated:

- Test-Driven Development using AI
- Writing tests before implementation
- Improved reliability and error handling
- Comparison of AI-generated tests with manual logic